It is important to note that the output is not deterministic. It will behave the same for a few elements, but race conditions apply. The larger the set, the more room to see the not-deterministic behavior of the overflow strategy. In this example, with this output, the first two elements are processed and then the buffer fills up, dropping the older elements until only the last ones remain.

The `queue.offer` method returns a value that you might be interested in. The following example shows two of the possible return values:

```
class QueueSpec ... {
  ...

  (1 to 10)
    .map(x => {
       queue.offer(x).map {
         case QueueOfferResult.Enqueued => println(s"enqueued $x")     ❶
         case QueueOfferResult.Dropped  => println(s"dropped $x")}})   ❷
  ...
}
```

❶ If offer returns Enqueued, prints “enqueued”

❷ If offer returns Dropped, prints “dropped”

If you change in the previous example to `OverflowStrategy.dropNew` and add the `queue.offer`, you get this:

```
enqueued 1
processed 2
enqueued 2
enqueued 3
enqueued 4
enqueued 5
enqueued 6
dropped 7
dropped 8
dropped 9
```